

1.5 Quicksort

Quicksort is another recursive sorting algorithm, discovered by Tony Hoare in 1959 and first published in 1961. In this algorithm, the hard work is splitting the array into small subarrays, and then sorting the sorted subarrays.

1. Choose a pivot element.
2. Partition the array into three subarrays containing the elements smaller than the pivot, the pivot element itself, and the elements larger than the pivot.
3. Recursively quicksort the first and last subarrays.

Figure 1.7. A diagram illustrating the partitioning step of the quicksort algorithm. The array is divided into three subarrays: elements smaller than the pivot, the pivot element itself, and elements larger than the pivot.

More detailed pseudocode is given in Figure 1.8. In the `partition` subroutine, the input parameter p is the index of the pivot element in the unsorted array; the subarray is partitioned into three subarrays, and the pivot element is placed in its final position. The `partition` subroutine returns the index of the pivot element. I'm pressing my finger on the `partition` subroutine in the pseudocode.

```

Quicksort(A, l, r)
  if (l > r)
    return
  Choose a pivot element A[p]
  r ← partition(A, l, r, p)
  Quicksort(A, l, r-1)
  Quicksort(A, r+1, r)
  
```

```

partition(A, l, r, p)
  for i ← l to r-1
    if A[i] < A[p]
      swap A[i] ↔ A[p-1]
  swap A[p-1] ↔ A[p]
  return p
  
```

Serie · Matemáticas para Machine Learning

Imagen y núcleo

Qué llega a cero y qué se puede alcanzar

smaller subarrays
so that recursion
array is smaller
1. Choose a pivot element from
the array.
2. Partition the array into three
subarrays containing the elements

Dos subespacios que trae Φ

Toda aplicación lineal $\Phi : V \rightarrow W$ carga consigo dos subespacios:

Def. El núcleo (o espacio nulo) son los vectores que Φ envía al cero; la imagen (o rango), los que Φ alcanza.

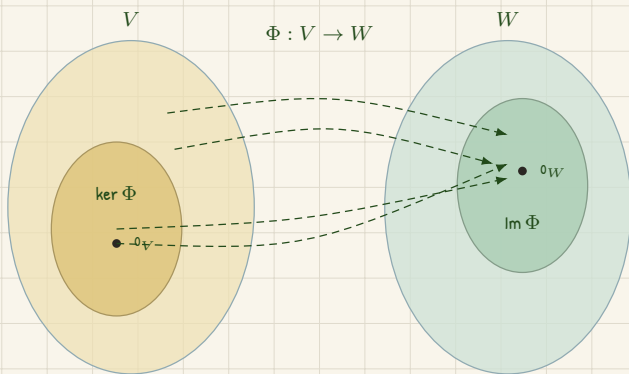
$$\ker(\Phi) = \{v \in V : \Phi(v) = 0\}$$

$$\text{Im}(\Phi) = \{\Phi(v) : v \in V\}$$

Obs. Llamamos dominio a V y codominio a W .



El mapa completo



Obs. Como $\Phi(0_V) = 0_W$, el 0 siempre está en el núcleo. Todo el $\ker \Phi$ colapsa al 0_W ; la $\text{Im } \Phi$ es lo que de verdad se alcanza.

Obs. Ambos son subespacios, y Φ es inyectiva exactamente cuando $\ker \Phi = \{0\}$.



Con una matriz

Si $\Phi(x) = Ax$ y A tiene columnas $a_1, \dots, a_n$, la imagen es su espacio columna:

$$\text{Im}(\Phi) = \text{span}[a_1, \dots, a_n]$$

Obs. Como $\text{rk}(A) = \dim(\text{Im } \Phi)$, la imagen mide justo el rango de A .

Obs. Y el núcleo es el conjunto solución de $Ax = 0$: codifica cómo dependen entre sí las columnas.



Un ejemplo

Sea $\Phi : \mathbb{R}^4 \rightarrow \mathbb{R}^2, x \mapsto Ax$ con $A = \begin{pmatrix} 1 & 2 & -1 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}$. La imagen es el span de sus columnas.

Para el núcleo, llevamos A a la forma reducida:

1	0	0	1
0	1	$-\frac{1}{2}$	$-\frac{1}{2}$

Sus dos columnas libres dan una base del núcleo (aquí $\dim \ker = 2$):

$$\ker(\Phi) = \text{span} \left[\begin{bmatrix} 0 \\ \frac{1}{2} \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} -1 \\ \frac{1}{2} \\ 0 \\ 1 \end{bmatrix} \right]$$



Rango y nulidad

El dominio se reparte exactamente entre lo que se aplasta y lo que se alcanza —el teorema fundamental de las aplicaciones lineales:

$$\dim(\ker \Phi) + \dim(\operatorname{Im} \Phi) = \dim(V)$$

Obs. Consecuencia elegante: si $\dim V = \dim W$, ser inyectiva, sobreyectiva o biyectiva es lo mismo.



¿Cuánto de tu espacio se pierde en el núcleo?

Forma parte de la serie

Matemáticas para Machine Learning.

Guarda esto y sígueme para las siguientes en asanchezyali.com

